

CSCE 649 PHYSICALLY-BASED MODELING · MODULE 13 · LECTURE NOTES

Shipping a Simulator

Determinism, time steps, sleeping, and the parts nobody writes papers about.

MODULE OUTCOMES

Decouple the simulation time step from the render frame rate.

Implement sleeping and explain the hysteresis it needs.

Achieve and maintain determinism.

Build the debugging tools a simulator requires.

Expose controls that an artist can actually use.

1 Time

NEVER STEP THE SIMULATION BY THE ELAPSED FRAME TIME

It is the obvious thing to do and it is wrong in three separate ways.

The physics becomes hardware-dependent. The same scene behaves differently on a fast machine and a slow one, because the integration error and the solver's convergence both depend on the step size. A bug reported by a user with different hardware cannot be reproduced.

Determinism is destroyed immediately (Module 01), and with it replays, networked lockstep, and reproducible bug reports.

Stability becomes a lottery. One unusually long frame — a texture load, a garbage collection, a breakpoint — produces a step that exceeds the stability limit and destroys the simulation.

```
const double DT = 1.0 / 120.0;
double accumulator = 0.0;

void frame(double real_elapsed) {
    accumulator += min(real_elapsed, 0.25);    // CLAMP: no death spiral
    while (accumulator >= DT) {
        prev_state = current_state;
        simulate(DT);
        accumulator -= DT;
    }
    double alpha = accumulator / DT;
    render(lerp(prev_state, current_state, alpha));    // INTERPOLATE
}
```

Detail	Why it is necessary
The clamp	Without it, a single very long frame — after a breakpoint, say — queues up hundreds of catch-up steps. Those take longer than real time, which makes the next frame longer still, which queues more steps. The 'spiral of death' : the application hangs and never recovers. Clamping accepts that time has been lost rather than trying to catch up.
The interpolation	The simulation rate and the display rate rarely divide evenly, so at any given frame the simulation is partway between two states. Rendering the most recent state produces visible judder — objects move in slightly uneven increments. Interpolating between the two most recent states by the leftover fraction removes it entirely, for the cost of storing one extra state.

2 Sleeping and islands

SLEEPING REQUIRES HYSTERESIS

A body whose velocity has remained below a threshold for several consecutive frames is **put to sleep**: removed from integration, from the broad phase's dynamic set, and from the constraint solver. A settled stack should cost nothing at all.

A single threshold oscillates. The body falls below it and sleeps; a neighbouring body's contact or an accumulated penetration correction nudges it awake; it settles and sleeps again. The wake-and-sleep churn costs more than never sleeping, and produces visible twitching.

So use two thresholds and a timer: sleep when velocity stays below a low threshold for N consecutive frames; wake immediately when it exceeds a higher one. The gap between them is the hysteresis.

And wake islands together. If any body in a connected contact group wakes, the entire group must wake — otherwise a box wakes and the crate it is resting on does not, so the box falls through it.

- **Group bodies connected by contacts or joints into islands** — a connected-components pass over the contact graph, which is cheap.
- **Solve each island independently.** By construction they do not interact during this step, so the solver works on a much smaller problem and converges faster for the same iteration count.
- **Sleep and wake per island**, which solves the problem above.
- **Solve islands in parallel.** They are genuinely independent, which makes this the cleanest parallelism available in a physics engine — no locks, no ordering concerns, no determinism risk if the per-island work is itself deterministic.
- **And a single large island defeats all three benefits.** One long chain of contacts — a big pile of objects all touching — makes everything a single island that cannot be split, cannot sleep partially, and cannot be parallelised. This is why a large pile is disproportionately slow, and it is a structural limit rather than an implementation problem.

3 Determinism

Cause	Remedy
Variable time step.	Fixed step with an accumulator (§1).
Iteration order over hash containers.	The order depends on insertion history and allocation addresses. Sort before iterating, or use ordered containers, or iterate over a stable index array.
Parallel reduction order.	Floating-point addition is not associative , so summing the same values in a different order gives a different answer. Use a fixed partitioning with a deterministic reduction tree rather than whatever order threads happen to finish in.
Uninitialised memory.	Zero-initialise everything and run under a sanitiser. This is the cause that takes longest to find.
Uncontrolled random seeds.	Seed explicitly and store the seed with the recording.
Cross-platform floating-point differences.	Genuinely hard. See below.

SCOPE THE GOAL TO SAME-MACHINE DETERMINISM

Same binary, same machine, same input, bit-identical output. This is achievable with the measures above, and it is what debugging actually requires: the ability to reproduce a failure exactly and to verify that a fix changed what you think it changed.

Cross-platform determinism is a different and much harder problem. Different compilers reorder floating-point operations differently; SIMD widths differ; fused multiply-add is available on some targets and not others, and changes results; and the implementations of transcendental functions are not specified to the last bit.

Lockstep networked games genuinely need it and pay substantially for it: fixed-point arithmetic throughout, software implementations of every transcendental, strict compiler flags that disable optimisations, and a great deal of testing.

For everything else, aim at same-machine determinism. It costs little and provides nearly all the debugging benefit.

4 Tools and controls

Tool	Value
Frame stepping, pausing, rewind.	Non-negotiable. The interesting frame is always the one before you noticed something was wrong.
Contact visualisation — points, normals, and impulse magnitudes drawn in the viewer.	The highest value per hour of implementation in this list. A wrong normal, a missing contact, or an enormous impulse is visible instantly and essentially invisible otherwise.

Tool	Value
Island and sleep-state visualisation by colour.	Makes the structure of §2 visible, and immediately shows when an island is larger than it should be.
Record and replay.	Capture the complete input stream so any failure can be reproduced exactly — which requires the determinism of §3.
Energy and momentum plots.	Module 01's harness, still the best diagnostic available.
A stress scene in CI that runs every commit and asserts invariants — no NaNs, bounded energy, the stack still standing.	Catches regressions that nobody would notice by eye until much later.

EXPOSE CONTROLS, NOT CONSTANTS

Module 01 argued that art-directability is a hard requirement. Concretely, that means these:

Per-object gravity scale. A cape that should settle faster than physics allows, a feather that should drift longer. One multiplier, and it is requested constantly.

Damping multipliers separate from material parameters, so motion can be calmed without changing what the material is.

Time-ranged forces. 'This wind acts on frames 40 to 90.' Completely unphysical and exactly what a shot requires.

Target shapes with adjustable attraction strength. The simulation is pulled toward a specified pose, which is how a simulation is made to hit a required silhouette at a required frame.

Caching and sub-frame output, so an expensive simulation is run once and reviewed, retimed, and rendered many times without rerunning it.

Where this course leaves you: able to express any physical system as a state vector and a derivative function, and to choose an integrator for a stated reason rather than by habit; able to distinguish an instability from a constraint failure from a collision failure, which is most of the practical skill; and clear about which methods in this field are derived from mechanics and which are not, and why the latter frequently win.

CSCE 647 renders what you have simulated here, **CSCE 641** is the pipeline it travels through, and **CSCE 650** adds a frame budget with a person inside it. And you have a simulator whose failure modes you can explain — which is the deliverable, because every simulator has them.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Glenn Fiedler — Fix Your Timestep! (free)	The accumulator pattern of §1, including the spiral of death and the interpolation. Short and definitive.
Glenn Fiedler — the rest of Gaffer On Games (free)	Determinism, networked physics, and lockstep. The practical counterpart to §3.
Erin Catto — Box2D source and GDC talks	Islands, sleeping, and the engineering of a shipped solver. Read <code>b2World::Solve</code> for the island construction of §2.
Bullet Physics manual	Production concerns: determinism caveats, sleeping thresholds, and debug drawing interfaces.
Christer Ericson — blog archive (free)	Numerical robustness and the practicalities of shipping physics code.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the fixed-step accumulator with interpolated rendering. Demonstrate judder with the interpolation removed.
- 2 Trigger the spiral of death by removing the clamp and stalling a frame. Then restore the clamp.
- 3 Run the same scenario at 30, 60, and 144 render frames per second and confirm the simulation result is identical.
- 4 Implement sleeping with a single threshold and demonstrate the oscillation. Then add hysteresis and a timer.
- 5 Implement simulation islands. Measure solve time for a scene of 1,000 separate objects against one of 1,000 objects in a single pile, and explain the difference.
- 6 Parallelise across islands and report the speedup. Verify the result is still bit-identical.
- 7 Achieve same-machine determinism and prove it: run a 10,000-step simulation twice and compare final states bit for bit.
- 8 Break it deliberately in each of the five ways in §3 and confirm each one shows up.
- 9 Implement contact visualisation. Then introduce a sign error in a contact normal and confirm you can see it immediately.
- 10 Build a stress scene and an invariant-checking harness, and run it as a test.
- 11 Add per-object gravity scale and a target-shape attractor. Use them to make a cloth simulation hit a specified pose at a specified frame.
- 12 **Project 2 is now due.** Submit the simulator, the videos, the energy or volume plots, the stability study, and the documented failure case.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give three reasons never to step the simulation by the frame time.
- 2 What does the accumulator clamp prevent, and what does the interpolation prevent?
- 3 Why does sleeping need two thresholds, and why must islands wake together?
- 4 Give four benefits of simulation islands and one case that defeats them all.
- 5 Name five causes of non-determinism and the remedy for each.
- 6 Why is floating-point summation order a determinism problem?
- 7 Why is cross-platform determinism so much harder, and who needs it?
- 8 Name four debugging tools and say which has the highest value per hour spent.
- 9 Give four unphysical controls a production simulator should expose, and say why.